

Page Object Model (POM) Layers: A Detailed Breakdown

The Page Object Model (POM) is more than just a pattern for creating page classes; it is a foundational design principle for a scalable, maintainable, and robust test automation framework. In a layered POM approach, each layer has a distinct responsibility, helping to organize the framework efficiently. This document provides a detailed breakdown of each layer, with Playwright and TypeScript as the reference implementation.

1. Page Layer (Core of POM)

This is the heart of the POM. It encapsulates all page-specific details, keeping them isolated from test logic.

Structure

- Create a dedicated directory: `pages/` or `pageObjects/`.
- Each file represents a single page (e.g., `LoginPage.ts`, `DashboardPage.ts`).
- Use meaningful class and file names for clarity.

Components of a Page Class

1.1 Class Definition

- Each page is represented as a TypeScript class.
- Pass the Playwright `Page` object via constructor to interact with the browser context.

```
import { Page } from '@playwright/test';
import { DashboardPage } from '../DashboardPage';

export class LoginPage {
  readonly page: Page;

  constructor(page: Page) {
    this.page = page;
  }
}
```

1.2 Locators (Properties)

- Use `page.locator()` or Playwright recommended locators (`getByRole`, `getByLabel`, etc.).
- Store locators as `readonly` properties at the top of the class.
- Avoid fragile selectors like generic CSS classes; prefer IDs, data-testid, or role-based selectors.

```
readonly usernameInput = this.page.getByLabel('Username');
readonly passwordInput = this.page.locator('#password');
readonly loginButton = this.page.getByRole('button', { name: 'Login' });
```

1.3 Methods (Actions)

- Perform actions like typing, clicking, or selecting.
- Name methods descriptively according to the user's intent (e.g., `login()`, `addProductToCart()`).
- Use method chaining: return a new Page object after navigation for fluent test code.

```
async login(username: string, password: string): Promise<DashboardPage> {  
  await this.usernameInput.fill(username);  
  await this.passwordInput.fill(password);  
  await this.loginButton.click();  
  return new DashboardPage(this.page); // returns next page  
}
```

Best Practices

- Keep methods assertion-free; assertions belong in the Test Layer.
- Encapsulate page behavior, not test logic.

2. Test Layer (The "What" to Test)

This layer contains the actual test cases, focusing purely on business logic and validation.

Structure

- Directory: `tests/`
- Test files: follow `.spec.ts` naming convention (`login.spec.ts`, `shopping.spec.ts`).
- Import Page Objects and `expect` from Playwright.

Responsibilities

- **Setup:** Use Playwright's test fixtures to obtain a `Page` object.
- **Instantiate Page Objects:** Create instances of pages required for the test.
- **Execute Test Steps:** Use page methods to perform actions.
- **Assertions:** Validate outcomes with Playwright `expect`.

```
import { test, expect } from '@playwright/test';  
import { LoginPage } from '../pages/LoginPage';  
import { DashboardPage } from '../pages/DashboardPage';  
  
test.describe('Login Functionality', () => {  
  test('should log in successfully with valid credentials', async ({  
    page }) => {  
    const loginPage = new LoginPage(page);  
  
    // Use Page Object method  
    const dashboardPage = await loginPage.login('standard_user',  
      'secret_sauce');
```

```
    // Perform assertions
    await expect(dashboardPage.productsHeader).toBeVisible();
    await expect(page).toHaveURL(/.*dashboard/);
  });
});
```

3. Data Layer (The "With What" to Test)

Separating test data from test code ensures maintainability and reusability.

Purpose

- Stores test data like user credentials, product details, or environment URLs.

Implementation

- Directory: `data/`
- Format: JSON (`testData.json`) for simplicity; Excel or CSV for complex datasets.
- Sensitive data: use environment variables or a secret management system.

```
{
  "validUser": {
    "username": "standard_user",
    "password": "secret_sauce"
  },
  "invalidUser": {
    "username": "locked_out_user",
    "password": "secret_sauce"
  }
}
```

Usage in Tests

```
import testData from '../data/testData.json';

test('should fail login with invalid credentials', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.login(testData.invalidUser.username,
testData.invalidUser.password);
  await expect(loginPage.errorMessage).toBeVisible();
});
```

4. Utility Layer (The "How" to Do Things)

Contains generic, reusable functions not tied to any page.

Examples

- **Data Helpers:** Read JSON/Excel, generate random strings.
 - **API Helpers:** Setup or cleanup test data via API.
 - **File Handlers:** Screenshot, file uploads/downloads.
 - **Logging:** Capture test execution details.
-

5. Reporting Layer

Provides visibility into test execution results.

Purpose

- Generate human-readable reports with pass/fail status, duration, and error details.

Implementation

- **Playwright HTML Reporter:** Built-in, simple, effective.
 - **Allure Reporter:** Detailed dashboards, step-by-step breakdowns, attach screenshots/logs.
-

6. Hooks Layer

Hooks manage the test lifecycle, ensuring consistent setup and teardown.

Key Hooks

- **beforeAll:** Runs once per test file (e.g., setup DB connection).
 - **beforeEach:** Runs before each test (e.g., navigate to base URL, create new Page object).
 - **afterEach:** Runs after each test (e.g., take screenshot on failure, cleanup).
 - **afterAll:** Runs once after all tests (e.g., global cleanup).
-

Summary

The layered POM approach separates concerns in a test automation framework:

Layer	Responsibility
Page Layer	Encapsulate page elements & actions
Test Layer	Define test scenarios & assertions
Data Layer	Store and manage test data

Layer	Responsibility
Utility Layer	Provide generic helper functions
Reporting Layer	Generate readable test reports
Hooks Layer	Manage test setup and teardown

This structure ensures readable, maintainable, scalable, and robust Playwright automation.